

template.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define rep(i, n)
4 using ll = long long;
5
6 void solve() {
7 }
8
9 signed main() {
10  cin.tie(0);
11  ios::sync_with_stdio(0);
12  cout << fixed << setprecision(20);
13  int TT;
14  TT = 1; //cin >> TT;
15  solve();
16 }

```

dsu.cpp

```

1 struct dsu {
2  vector<int> p;
3  dsu(int n) : p(n, -1) {}
4  int leader(int x) {
5      return p[x] < 0 ? x
6          : p[x] = leader(p[x]);
7  }
8  int merge(int a, int b) {
9      a = leader(a);
10     b = leader(b);
11     if (a == b)
12         return a;
13     if (p[a] > p[b])
14         swap(a, b);
15     p[a] += p[b];
16     p[b] = a;
17     return a;
18 }
19 bool same(int a, int b) {
20     return leader(a) == leader(b);
21 }
22 int size(int a) { return -p[leader(a)]; }
23 };

```

segtree.cpp

```

1 template <class S, S op(S, S), S e()>
2 struct segtree {
3  int n, sz;
4  vector<S> d;
5  segtree(int m = 0) : n(1), sz(m) {
6      while (n < m)
7          n *= 2;
8      d.assign(2 * n, e());
9  }
10 segtree(vector<S> a) : segtree(a.size()) {
11     copy(a.begin(), a.end(), d.begin() + n);
12     for (int i = n - 1; i > 0; i--)
13         d[i] = op(d[2 * i], d[2 * i + 1]);
14 }
15 void set(int p, S x) {
16     for (d[p += n] = x; p /= 2; )
17         d[p] = op(d[2 * p], d[2 * p + 1]);
18 }
19 S get(int p) { return d[p + n]; }
20 S prod(int l, int r) {
21     S x = e(), y = e();
22     for (l += n, r += n; l < r;
23          l /= 2, r /= 2) {
24         if (l & 1)
25             x = op(x, d[l++]);

```

segtree.cpp (cont.)

```

26         if (r & 1)
27             y = op(d[--r], y);
28     }
29     return op(x, y);
30 }
31 S all_prod() { return d[1]; }
32
33 template <class G>
34 int max_right(int l, G g) {
35     assert(g(e()));
36     S x = e();
37     int r = mr(l, g, x, 1, 0, n);
38     return r < 0 ? sz : r;
39 }
40 template <class G>
41 int mr(int a, G& g, S& x, int k, int l,
42        int r) {
43     if (r <= a || sz <= l)
44         return -1;
45     if (a <= l && r <= sz) {
46         S y = op(x, d[k]);
47         if (g(y)) {
48             x = y;
49             return -1;
50         }
51         if (r - l == 1)
52             return l;
53     }
54     int m = (l + r) / 2,
55         q = mr(a, g, x, 2 * k, l, m);
56     return q < 0
57         ? mr(a, g, x, 2 * k + 1, m, r)
58         : q;
59 }
60 template <class G>
61 int min_left(int r, G g) {
62     assert(g(e()));
63     S x = e();
64     int l = ml(r, g, x, 1, 0, n);
65     return l < 0 ? 0 : l;
66 }
67 template <class G>
68 int ml(int b, G& g, S& x, int k, int l,
69        int r) {
70     if (b <= l)
71         return -1;
72     if (r <= b) {
73         S y = op(d[k], x);
74         if (g(y)) {
75             x = y;
76             return -1;
77         }
78         if (r - l == 1)
79             return r;
80     }
81     int m = (l + r) / 2,
82         q = ml(b, g, x, 2 * k + 1, m, r);
83     return q < 0 ? ml(b, g, x, 2 * k, l, m)
84         : q;
85 }
86 };

```

lazy_segtree.cpp

```

1 template <class S, S op(S, S), S e(),
2         class F, S mapping(F, S),
3         F composition(F, F), F id()>

```

lazy_segtree.cpp (cont.)

```

4 struct lazy_segtree {
5     int n, sz;
6     vector<S> d;
7     vector<F> lz;
8     lazy_segtree(int m = 0) : n(1), sz(m) {
9         while (n < m)
10             n *= 2;
11         d.assign(2 * n, e());
12         lz.assign(2 * n, id());
13     }
14     lazy_segtree(vector<S> a)
15         : lazy_segtree(a.size()) {
16         copy(a.begin(), a.end(), d.begin() + n);
17         for (int i = n - 1; i; i--)
18             d[i] = op(d[2 * i], d[2 * i + 1]);
19     }
20     void all(int k, F f) {
21         d[k] = mapping(f, d[k]);
22         lz[k] = composition(f, lz[k]);
23     }
24     void push(int k) {
25         all(2 * k, lz[k]);
26         all(2 * k + 1, lz[k]);
27         lz[k] = id();
28     }
29     void set(int p, S x) {
30         set(p, x, 1, 0, n);
31     }
32     void set(
33         int p, S x, int k, int l, int r) {
34         if (r - l == 1) {
35             d[k] = x;
36             lz[k] = id();
37             return;
38         }
39         push(k);
40         int m = (l + r) / 2;
41         p < m ? set(p, x, 2 * k, l, m)
42             : set(p, x, 2 * k + 1, m, r);
43         d[k] = op(d[2 * k], d[2 * k + 1]);
44     }
45     S get(int p) { return prod(p, p + 1); }
46     S prod(int a, int b) {
47         return prod(a, b, 1, 0, n);
48     }
49     S prod(
50         int a, int b, int k, int l, int r) {
51         if (r <= a || b <= l)
52             return e();
53         if (a <= l && r <= b)
54             return d[k];
55         push(k);
56         int m = (l + r) / 2;
57         return op(prod(a, b, 2 * k, l, m),
58             prod(a, b, 2 * k + 1, m, r));
59     }
60     S all_prod() { return d[1]; }
61     void apply(int p, F f) {
62         apply(p, p + 1, f);
63     }
64     void apply(int a, int b, F f) {
65         apply(a, b, f, 1, 0, n);
66     }
67     void apply(int a, int b, F f, int k,
68         int l, int r) {
69         if (r <= a || b <= l)
70             return;

```

lazy_segtree.cpp (cont.)

```

71     if (a <= l && r <= b) {
72         all(k, f);
73         return;
74     }
75     push(k);
76     int m = (l + r) / 2;
77     apply(a, b, f, 2 * k, l, m);
78     apply(a, b, f, 2 * k + 1, m, r);
79     d[k] = op(d[2 * k], d[2 * k + 1]);
80 }
81
82 template <class G>
83 int max_right(int l, G g) {
84     assert(g(e()));
85     S x = e();
86     int r = mr(l, g, x, 1, 0, n);
87     return r < 0 ? sz : r;
88 }
89 template <class G>
90 int mr(int a, G& g, S& x, int k, int l,
91     int r) {
92     if (r <= a || sz <= l)
93         return -1;
94     if (a <= l && r <= sz) {
95         S y = op(x, d[k]);
96         if (g(y)) {
97             x = y;
98             return -1;
99         }
100         if (r - l == 1)
101             return l;
102     }
103     push(k);
104     int m = (l + r) / 2;
105     int q = mr(a, g, x, 2 * k, l, m);
106     return q < 0
107         ? mr(a, g, x, 2 * k + 1, m, r)
108         : q;
109 }
110 template <class G>
111 int min_left(int r, G g) {
112     assert(g(e()));
113     S x = e();
114     int l = ml(r, g, x, 1, 0, n);
115     return l < 0 ? 0 : l;
116 }
117 template <class G>
118 int ml(int b, G& g, S& x, int k, int l,
119     int r) {
120     if (b <= l)
121         return -1;
122     if (r <= b) {
123         S y = op(d[k], x);
124         if (g(y)) {
125             x = y;
126             return -1;
127         }
128         if (r - l == 1)
129             return r;
130     }
131     push(k);
132     int m = (l + r) / 2;
133     int q = ml(b, g, x, 2 * k + 1, m, r);
134     return q < 0 ? ml(b, g, x, 2 * k, l, m)
135         : q;
136 }
137 };

```

lazy_segtree.cpp (cont.)

```

138
139 template <class S, S op(S, S), S e()>
140 struct segtree {
141     int n, sz;
142     vector<S> d;
143     segtree(int m = 0) : n(1), sz(m) {
144         while (n < m)
145             n *= 2;
146         d.assign(2 * n, e());
147     }
148     segtree(vector<S> a) : segtree(a.size()) {
149         copy(a.begin(), a.end(), d.begin() + n);
150         for (int i = n - 1; i; i--)
151             d[i] = op(d[2 * i], d[2 * i + 1]);
152     }
153     void set(int p, S x) {
154         for (d[p += n] = x; p /= 2;)
155             d[p] = op(d[2 * p], d[2 * p + 1]);
156     }
157     S get(int p) { return d[p + n]; }
158     S prod(int l, int r) {
159         S x = e(), y = e();
160         for (l += n, r += n; l < r;
161              l /= 2, r /= 2) {
162             if (l & 1)
163                 x = op(x, d[l++]);
164             if (r & 1)
165                 y = op(d[--r], y);
166         }
167         return op(x, y);
168     }
169     S all_prod() { return d[1]; }
170
171     template <class G>
172     int max_right(int l, G g) {
173         assert(g(e()));
174         S x = e();
175         int r = mr(l, g, x, 1, 0, n);
176         return r < 0 ? sz : r;
177     }
178     template <class G>
179     int mr(int a, G& g, S& x, int k, int l,
180            int r) {
181         if (r <= a || sz <= l)
182             return -1;
183         if (a <= l && r <= sz) {
184             S y = op(x, d[k]);
185             if (g(y)) {
186                 x = y;
187                 return -1;
188             }
189             if (r - l == 1)
190                 return l;
191         }
192         int m = (l + r) / 2,
193             q = mr(a, g, x, 2 * k, l, m);
194         return q < 0
195             ? mr(a, g, x, 2 * k + 1, m, r)
196             : q;
197     }
198     template <class G>
199     int min_left(int r, G g) {
200         assert(g(e()));
201         S x = e();
202         int l = ml(r, g, x, 1, 0, n);
203         return l < 0 ? 0 : l;
204     }

```

lazy_segtree.cpp (cont.)

```

205     template <class G>
206     int ml(int b, G& g, S& x, int k, int l,
207            int r) {
208         if (b <= l)
209             return -1;
210         if (r <= b) {
211             S y = op(d[k], x);
212             if (g(y)) {
213                 x = y;
214                 return -1;
215             }
216             if (r - l == 1)
217                 return r;
218         }
219         int m = (l + r) / 2,
220             q = ml(b, g, x, 2 * k + 1, m, r);
221         return q < 0 ? ml(b, g, x, 2 * k, l, m)
222             : q;
223     }
224 };

```

binarytrie.cpp

```

1 template <int B = 30>
2 struct binary_trie {
3     struct N {
4         int nx[2] = {-1, -1}, cnt = 0;
5     };
6     vector<N> t{N()};
7     void add(long long x, int d = 1) {
8         int v = 0;
9         t[v].cnt += d;
10        for (int i = B - 1; i >= 0; i--) {
11            int b = x >> i & 1;
12            if (t[v].nx[b] < 0)
13                t[v].nx[b] = t.size(),
14                t.emplace_back();
15            v = t[v].nx[b];
16            t[v].cnt += d;
17        }
18    }
19    void insert(long long x) { add(x, 1); }
20    void erase(long long x) { add(x, -1); }
21    long long min_xor(long long x) {
22        int v = 0;
23        long long r = 0;
24        for (int i = B - 1; i >= 0; i--) {
25            int b = x >> i & 1, u = t[v].nx[b];
26            if (u < 0 || !t[u].cnt)
27                b ^= 1, r |= 1LL << i;
28            v = t[v].nx[b];
29        }
30        return r;
31    }
32    long long max_xor(long long x) {
33        int v = 0;
34        long long r = 0;
35        for (int i = B - 1; i >= 0; i--) {
36            int b = x >> i & 1,
37                u = t[v].nx[b ^ 1];
38            if (u >= 0 && t[u].cnt)
39                b ^= 1, r |= 1LL << i;
40            v = t[v].nx[b];
41        }
42        return r;
43    }
44 };

```

trie.cpp

```

1 template <int K, char B>
2 struct trie { //文字種類数 / 最初の文字
3     struct N {
4         int nx[K], end = 0;
5         N() { fill(nx, nx + K, -1); }
6     };
7     vector<N> t{N()};
8     void insert(string s) {
9         int v = 0;
10        for (char c : s) {
11            int x = c -
12                B; //x が id にあたる 連続でないなら id(
13                c)に変更
14            if (t[v].nx[x] < 0)
15                t[v].nx[x] = t.size(),
16                t.emplace_back();
17            v = t[v].nx[x];
18        }
19        t[v].end++;
20    }
21    int find(string s) {
22        int v = 0;
23        for (char c : s) {
24            v = t[v].nx[c - B];
25            if (v < 0)
26                return -1;
27        }
28        return v;
29    };

```

suffix_array.cpp

```

1 template <class T>
2 vector<int> suffix_array(vector<T> s) {
3     int n = s.size();
4     if (!n)
5         return {};
6     int N = n + 1;
7     vector<int> p(N), c(N), q(N), nc(N),
8         cnt(N);
9     p[0] = n;
10    iota(p.begin() + 1, p.end(), 0);
11    sort(p.begin() + 1, p.end(),
12        [&](int i, int j) {
13            return s[i] < s[j];
14        });
15    for (int i = 1; i < N; i++)
16        c[p[i]] = i == 1 ? 1
17            : c[p[i - 1]] +
18            (s[p[i - 1]] != s[p[i]]);
19    for (int k = 1; k < N; k *= 2) {
20        for (int i = 0; i < N; i++)
21            q[i] = (p[i] - k + N) % N;
22        fill(cnt.begin(), cnt.end(), 0);
23        for (int x : q)
24            cnt[c[x]]++;
25        for (int i = 1; i < N; i++)
26            cnt[i] += cnt[i - 1];
27        for (int i = N - 1; i >= 0; i--)
28            p[--cnt[c[q[i]]]] = q[i];
29        nc[p[0]] = 0;
30        for (int i = 1; i < N; i++)
31            nc[p[i]] = nc[p[i - 1]] +
32                (c[p[i - 1]] != c[p[i]] ||
33                 c[(p[i - 1] + k) % N] !=
34                 c[(p[i] + k) % N]);

```

suffix_array.cpp (cont.)

```

35    c.swap(nc);
36 }
37 return {p.begin() + 1, p.end()};
38 }
39 vector<int> suffix_array(string s) {
40     return suffix_array(vector<unsigned char>(
41         s.begin(), s.end()));
42 }
43 vector<int> suffix_array(
44     vector<int> s, int) {
45     return suffix_array<int>(move(s));
46 }

```

lcp_array.cpp

```

1 template <class T>
2 vector<int> lcp_array(
3     const T& s, const vector<int>& sa) {
4     int n = s.size(), h = 0;
5     vector<int> r(n), l(n - 1);
6     for (int i = 0; i < n; i++)
7         r[sa[i]] = i;
8     for (int i = 0; i < n; i++)
9         if (r[i]) {
10            int j = sa[r[i] - 1];
11            while (max(i, j) + h < n &&
12                s[i + h] == s[j + h])
13                h++;
14            l[r[i] - 1] = h;
15            if (h)
16                h--;
17        }
18    return l;
19 }

```

z_algorithm.cpp

```

1 template <class T>
2 vector<int> z_algorithm(const T& s) {
3     int n = s.size();
4     vector<int> z(n);
5     for (int i = 1, j = 0; i < n; i++) {
6         if (i < j + z[j])
7             z[i] = min(j + z[j] - i, z[i - j]);
8         while (i + z[i] < n &&
9             s[z[i]] == s[i + z[i]])
10            z[i]++;
11        if (i + z[i] > j + z[j])
12            j = i;
13    }
14    if (n)
15        z[0] = n;
16    return z;
17 }

```

rolling_hash.cpp

```

1 struct rolling_hash {
2     using ull = unsigned long long;
3     using u128 = __uint128_t;
4     static constexpr ull M = (1ULL << 61) - 1;
5     static ull mul(ull a, ull b) {
6         u128 x = (u128)a * b;
7         ull y = (x >> 61) + (x & M);
8         return y >= M ? y - M : y;
9     }
10    inline static ull B =
11        chrono::steady_clock::now()
12            .time_since_epoch()
13            .count() %
14        (M - 256) +

```

rolling_hash.cpp (cont.)

```

15     256;
16     vector<ull> h, p;
17     rolling_hash(const string& s)
18         : h(s.size() + 1),
19           p(s.size() + 1, 1) {
20         for (int i = 0; i < (int)s.size(); i++)
21             p[i + 1] = mul(p[i], B),
22             h[i + 1] =
23                 (mul(h[i], B) +
24                  (unsigned char)s[i] + 1) %
25                 M;
26     }
27     ull get(int l, int r) const {
28         ull x = h[r] + M - mul(h[l], p[r - l]);
29         return x >= M ? x - M : x;
30     }
31 };

```

scc.cpp

```

1 struct scc_graph {
2     int n;
3     vector<vector<int>> g, rg;
4     scc_graph(int n) : n(n), g(n), rg(n) {}
5     void add_edge(int a, int b) {
6         g[a].push_back(b);
7         rg[b].push_back(a);
8     }
9     vector<vector<int>> scc() {
10        vector<int> vis(n), ord;
11        for (int s = 0; s < n; s++)
12            if (!vis[s]) {
13                stack<pair<int, int>> q;
14                q.push({s, 0});
15                vis[s] = 1;
16                while (q.size()) {
17                    auto& [v, i] = q.top();
18                    if (i < (int)g[v].size()) {
19                        int u = g[v][i++];
20                        if (!vis[u])
21                            vis[u] = 1, q.push({u, 0});
22                    } else
23                        ord.push_back(v), q.pop();
24                }
25            }
26        fill(vis.begin(), vis.end(), 0);
27        reverse(ord.begin(), ord.end());
28        vector<vector<int>> res;
29        for (int s : ord)
30            if (!vis[s]) {
31                res.push_back({});
32                stack<int> q;
33                q.push(s);
34                vis[s] = 1;
35                while (q.size()) {
36                    int v = q.top();
37                    q.pop();
38                    res.back().push_back(v);
39                    for (int u : rg[v])
40                        if (!vis[u])
41                            vis[u] = 1, q.push(u);
42                }
43            }
44        return res;
45    }
46 };

```

twosat.cpp

```

1 struct two_sat {
2     int n;
3     vector<vector<int>> g, rg;
4     vector<bool> a;
5     two_sat(int N)
6         : n(N), g(2 * N), rg(2 * N), a(N) {}
7
8     void add(int x, int y) {
9         g[x].push_back(y);
10        rg[y].push_back(x);
11    }
12    void add_clause(
13        int i, bool f, int j, bool h) {
14        add(2 * i + !f, 2 * j + h);
15        add(2 * j + !h, 2 * i + f);
16    }
17
18    bool satisfiable() {
19        int N = 2 * n;
20        vector<int> v(N), o, c(N);
21        auto dfs = [&](auto&& self,
22            int x) -> void {
23            v[x] = 1;
24            for (int y : g[x])
25                if (!v[y])
26                    self(self, y);
27            o.push_back(x);
28        };
29        for (int i = 0; i < N; i++)
30            if (!v[i])
31                dfs(dfs, i);
32
33        fill(v.begin(), v.end(), 0);
34        reverse(o.begin(), o.end());
35        int k = 0;
36        auto rdfs = [&](auto&& self,
37            int x) -> void {
38            v[x] = 1;
39            c[x] = k;
40            for (int y : rg[x])
41                if (!v[y])
42                    self(self, y);
43        };
44        for (int x : o)
45            if (!v[x])
46                rdfs(rdfs, x), k++;
47
48        for (int i = 0; i < n; i++) {
49            if (c[2 * i] == c[2 * i + 1])
50                return false;
51            a[i] = c[2 * i] < c[2 * i + 1];
52        }
53        return true;
54    }
55
56    vector<bool> answer() { return a; }
57 };

```

rerooting.cpp

```

1 /*
2  <Rerooting(全方位木 DP)>
3  - n 頂点の木 T の各頂点を値とした時の木 DP の
4    結果を得る
5
6  [実装/関数]
7  (構造体)

```

rerooting.cpp (cont.)

```

7      - struct RerootingDP<class E, class
↳ V, merge, e, put_edge, put_vertex> dp(n)
8          - E : 辺に持たせる情報の型
9          - V : 頂点に持たせる情報の型(
↳ ≠頂点の番号) 各頂点の dp 表にはこの型の値を
↳ 記録する
10         - E merge(E e1, E e2) :
↳ 同じ頂点を親側に持つ二つの辺をマージして新
↳ たな辺の情報として持つ
11         - E e() : merge の単位元
12         - E put_edge(V v, int i) :
↳ 頂点 i の親側に辺を 1 つ張った時, その辺が持つ
↳ 情報を計算
13         - V put_vertex(E e, int i) :
↳ 頂点 i の全ての子側の辺をマージした値から,
↳ 頂点の情報を計算
14         - 木の葉の dp の値は, E の単位
↳ 元 e によって put_vertex(e,i) により定まる
15         (葉 : put_vertex(e,i) により生成
↳ 他 : 葉の値をそれぞれ put_edge
↳ -> merge -> 得られた値を put_vertex)
16
17         (関数)
18         - void add_edge(int u, int v, int i,
↳ int j) :
20         頂点 u から v への番号 i の辺と, 頂点 v
↳ から u への番号 j の辺を追加する(i=j でもよい)
21
22         - V build(int v) : 頂点 v を根とした時
↳ の dp[v] の値を返す(1 回しか呼べない)
23
24         - vector<V> reroot() : 頂点 i を根とし
↳ た時の dp[i] の値を持つ配列を返す
25         (事前に build をちょうど 1 回呼んで
↳ いる必要あり)
26
27         [計算時間]
28         - コンストラクタ 0(n), build 0(n),
↳ reroot 0(n)
29
30         [備考]
31         - (codeforces の G++ 20 では E として pii
↳ を用いるとうまく動かなかった G++ 17
↳ を用いること)
32         - put_edge と put_vertex の添字 i の型は i
↳ nt で指定すること(!! とするとコンパイルエラ
↳ ー)
33
34         [参照]
35         - traP
36
37         [verified at]
38         - https://atcoder.jp/contests/abc394
↳ /submissions/63066845
39
40 */
41
42 /*
43     - n 頂点の木 T が与えられている
44
45     - RerootingDP<E,V,merge,e,put_edge,
↳ put_vertex> g(n) : コンストラクタ 0(n)
46     E : 親の dp の計算をする際に用いる値の
↳ 型
47     V : 頂点の dp の値の型

```

rerooting.cpp (cont.)

```

48     E put_edge (V v, int i) :
↳ 頂点 v を根とする部分木の dp の値に辺 i を追加し
↳ た時に得られる値
49     E merge(E e1, E e2) : 2 つの子につい
↳ て求めた put_edge の値を結合する
50     merge は可換モノイドによる結合を
↳ 要求される
51     E e() : merge の単位元を返す関数
52     V put_vertex(E e, int u) :
↳ 全ての子について merge した値から, 頂点 u の dp
↳ [u] を計算する
53     - void add_edge(int u, int v, int idx,
↳ int xdi) : 頂点 u->v の辺番号 idx の辺と
54     頂点 v->u の辺番号 xdi の辺を追加する (
↳ ちょうど n-1 回呼ぶ, ならし 0(1))
55     - V bulid(int v): 頂点 v を根とした時の, d
↳ p[v] の値を返す (1 回までしか呼べない 0(n))
56     - vector<V> reroot(): 長さ n の配列(
↳ i 番目には, 頂点 i を根とした時の, dp[
↳ i] の値) を返す
57     (先に build をちょうど 1 回呼んでいる必
↳ 要あり 0(n))
58 */
59
60 template <class E, class V,
61 E (*merge)(E, E), E (*e)(),
62 E (*put_edge)(V, int),
63 V (*put_vertex)(E, int)>
64 struct RerootingDP {
65     struct edge {
66         int to, idx, xdi;
67     };
68     RerootingDP(int n_ = 0)
69         : n(n_), inner_edge_id(0) {
70         es.resize(2 * n - 2);
71         start.resize(2 * n - 2);
72         if (n == 1)
73             es_build();
74     }
75     void add_edge(
76         int u, int v, int idx, int xdi) {
77         start[inner_edge_id] = u;
78         es[inner_edge_id] = {v, idx, xdi};
79         inner_edge_id++;
80         start[inner_edge_id] = v;
81         es[inner_edge_id] = {u, xdi, idx};
82         inner_edge_id++;
83         if (inner_edge_id == 2 * n - 2) {
84             es_build();
85         }
86     }
87     vector<V> build(int root_ = 0) {
88         root = root_;
89         vector<V> subdp(n);
90         subdp[0] = put_vertex(e(), 0);
91         outs.resize(n);
92         vector<int> geta(n + 1, 0);
93         for (int i = 0; i < n; i++)
94             geta[i + 1] =
95                 start[i + 1] - start[i] - 1;
96         geta[root + 1]++;
97         for (int i = 0; i < n; i++)
98             geta[i + 1] += geta[i];
99         auto dfs = [&](auto sfs, int v,
100             int f) -> void {
101             E val = e();

```


rerooting.cpp (cont.)

```

102     for (int i = start[v];
103          i < start[v + 1]; i++) {
104         if (es[i].to == f) {
105             swap(es[start[v + 1] - 1], es[i]);
106         }
107         if (es[i].to == f)
108             continue;
109         sfs(sfs, es[i].to, v);
110         E nval = put_edge(
111             subdp[es[i].to], es[i].idx);
112         outs[geta[v]++] = nval;
113         val = merge(val, nval);
114     }
115     subdp[v] = put_vertex(val, v);
116 };
117 dfs(dfs, root, -1);
118 return subdp;
119 }
120 vector<V> reroot() {
121     vector<E> reverse_edge(n);
122     reverse_edge[root] = e();
123     vector<V> answers(n);
124     auto dfs = [&](
125         auto sfs, int v) -> void {
126         int le = outs_start(v);
127         int ri = outs_start(v + 1);
128         int siz = ri - le;
129         vector<E> rui(siz + 1);
130         rui[siz] = e();
131         for (int i = siz - 1; i >= 0; i--) {
132             rui[i] =
133                 merge(outs[le + i], rui[i + 1]);
134         }
135         answers[v] = put_vertex(
136             merge(rui[0], reverse_edge[v]), v);
137         E lui = e();
138         for (int i = 0; i < siz; i++) {
139             V rdp = put_vertex(
140                 merge(merge(lui, rui[i + 1]),
141                     reverse_edge[v]),
142                 v);
143             reverse_edge[es[start[v] + i].to] =
144                 put_edge(
145                     rdp, es[start[v] + i].xdi);
146             lui = merge(lui, outs[le + i]);
147             sfs(sfs, es[start[v] + i].to);
148         }
149     };
150     dfs(dfs, root);
151     return answers;
152 }
153
154 private:
155 int n, root, inner_edge_id;
156 vector<E> outs;
157 vector<edge> es;
158 vector<int> start;
159 int outs_start(int v) {
160     int res = start[v] - v;
161     if (root < v)
162         res++;
163     return res;
164 }
165 void es_build() {
166     vector<edge> nes(2 * n - 2);
167     vector<int> nstart(n + 2, 0);
168     for (int i = 0; i < 2 * n - 2; i++)

```

rerooting.cpp (cont.)

```

169     nstart[start[i] + 2]++;
170     for (int i = 0; i < n; i++)
171         nstart[i + 1] += nstart[i];
172     for (int i = 0; i < 2 * n - 2; i++)
173         nes[nstart[start[i] + 1]++] = es[i];
174     swap(es, nes);
175     swap(start, nstart);
176 }
177 };

```

maxflow.cpp

```

1 template <class Cap>
2 struct mf_graph {
3     struct E {
4         int to, rev;
5         Cap cap;
6     };
7     struct edge {
8         int from, to;
9         Cap cap, flow;
10    };
11    int N;
12    vector<vector<E>> g;
13    vector<pair<int, int>> pos;
14
15    mf_graph(int n = 0) : N(n), g(n) {}
16
17    int add_edge(int s, int t, Cap c) {
18        int a = g[s].size(),
19            b = g[t].size() + (s == t);
20        pos.push_back({s, a});
21        g[s].push_back({t, b, c});
22        g[t].push_back({s, a, 0});
23        return (int)pos.size() - 1;
24    }
25
26    edge get_edge(int i) {
27        auto [v, j] = pos[i];
28        auto &e = g[v][j], &r = g[e.to][e.rev];
29        return {v, e.to, e.cap + r.cap, r.cap};
30    }
31
32    vector<edge> edges() {
33        vector<edge> r;
34        for (int i = 0; i < (int)pos.size();
35             i++)
36            r.push_back(get_edge(i));
37        return r;
38    }
39
40    void change_edge(int i, Cap c, Cap f) {
41        auto [v, j] = pos[i];
42        auto &e = g[v][j], &r = g[e.to][e.rev];
43        e.cap = c - f;
44        r.cap = f;
45    }
46
47    Cap flow(int s, int t) {
48        return flow(
49            s, t, numeric_limits<Cap>::max());
50    }
51
52    Cap flow(int s, int t, Cap lim) {
53        Cap ans = 0;
54        vector<int> lv(N), it(N);

```

maxflow.cpp (cont.)

```

55 while (ans < lim) {
56     fill(lv.begin(), lv.end(), -1);
57     queue<int> q;
58     q.push(s);
59     lv[s] = 0;
60     while (q.size()) {
61         int v = q.front();
62         q.pop();
63         for (auto& e : g[v])
64             if (e.cap && lv[e.to] < 0)
65                 lv[e.to] = lv[v] + 1,
66                 q.push(e.to);
67     }
68     if (lv[t] < 0)
69         break;
70     fill(it.begin(), it.end(), 0);
71
72     auto dfs = [&](auto&& self, int v,
73                 Cap f) -> Cap {
74         if (v == t)
75             return f;
76         for (int& i = it[v];
77              i < (int)g[v].size(); i++) {
78             E& e = g[v][i];
79             if (e.cap &&
80                 lv[e.to] == lv[v] + 1)
81                 if (Cap x = self(self, e.to,
82                                 min(f, e.cap))) {
83                     e.cap -= x;
84                     g[e.to][e.rev].cap += x;
85                     return x;
86                 }
87         }
88         return 0;
89     };
90
91     while (ans < lim) {
92         Cap x = dfs(dfs, s, lim - ans);
93         if (!x)
94             break;
95         ans += x;
96     }
97 }
98 return ans;
99 }
100
101 vector<bool> min_cut(int s) {
102     vector<bool> z(N);
103     queue<int> q;
104     q.push(s);
105     z[s] = 1;
106     while (q.size()) {
107         int v = q.front();
108         q.pop();
109         for (auto& e : g[v])
110             if (e.cap && !z[e.to])
111                 z[e.to] = 1, q.push(e.to);
112     }
113     return z;
114 }
115 };

```

mincostflow.cpp

```

1 template <class Cap, class Cost>
2 struct mcf_graph {

```

mincostflow.cpp (cont.)

```

3 struct E {
4     int to, rev;
5     Cap cap;
6     Cost cost;
7 };
8 struct edge {
9     int from, to;
10    Cap cap, flow;
11    Cost cost;
12 };
13 int N;
14 vector<vector<E>> g;
15 vector<pair<int, int>> pos;
16
17 mcf_graph(int n = 0) : N(n), g(n) {}
18
19 int add_edge(
20     int s, int t, Cap c, Cost w) {
21     int a = g[s].size(),
22         b = g[t].size() + (s == t);
23     pos.push_back({s, a});
24     g[s].push_back({t, b, c, w});
25     g[t].push_back({s, a, 0, -w});
26     return (int)pos.size() - 1;
27 }
28
29 edge get_edge(int i) {
30     auto [v, j] = pos[i];
31     auto &e = g[v][j], &r = g[e.to][e.rev];
32     return {v, e.to, e.cap + r.cap, r.cap,
33             e.cost};
34 }
35
36 vector<edge> edges() {
37     vector<edge> r;
38     for (int i = 0; i < (int)pos.size();
39          i++)
40         r.push_back(get_edge(i));
41     return r;
42 }
43
44 pair<Cap, Cost> flow(int s, int t) {
45     return flow(
46         s, t, numeric_limits<Cap>::max());
47 }
48
49 pair<Cap, Cost> flow(
50     int s, int t, Cap lim) {
51     return slope(s, t, lim).back();
52 }
53
54 vector<pair<Cap, Cost>> slope(
55     int s, int t) {
56     return slope(
57         s, t, numeric_limits<Cap>::max());
58 }
59
60 vector<pair<Cap, Cost>> slope(
61     int s, int t, Cap lim) {
62     const Cost INF =
63         numeric_limits<Cost>::max();
64     vector<Cost> h(N), d(N);
65     vector<int> pv(N), pe(N);
66
67     Cap f = 0;
68     Cost c = 0, last = -1;
69     vector<pair<Cap, Cost>> res{{0, 0}};

```


mincostflow.cpp (cont.)

```

70
71 while (f < lim) {
72     fill(d.begin(), d.end(), INF);
73     d[s] = 0;
74
75     priority_queue<pair<Cost, int>,
76         vector<pair<Cost, int>>,
77         greater<pair<Cost, int>>>
78         q;
79     q.push({0, s});
80
81     while (q.size()) {
82         auto [dv, v] = q.top();
83         q.pop();
84         if (dv != d[v])
85             continue;
86
87         for (int i = 0;
88             i < (int)g[v].size(); i++) {
89             E& e = g[v][i];
90             if (!e.cap)
91                 continue;
92
93             __int128 nd = (__int128)dv +
94                 e.cost + h[v] - h[e.to];
95             if (nd < d[e.to]) {
96                 d[e.to] = (Cost)nd;
97                 pv[e.to] = v;
98                 pe[e.to] = i;
99                 q.push({(Cost)nd, e.to});
100             }
101         }
102     }
103
104     if (d[t] == INF)
105         break;
106
107     for (int v = 0; v < N; v++)
108         if (d[v] < INF)
109             h[v] += d[v];
110
111     Cost x = lim - f;
112     for (int v = t; v != s; v = pv[v])
113         x = min(x, g[pv[v]][pe[v]].cap);
114
115     for (int v = t; v != s; v = pv[v]) {
116         E& e = g[pv[v]][pe[v]];
117         e.cap -= x;
118         g[v][e.rev].cap += x;
119     }
120
121     Cost w = h[t] - h[s];
122     f += x;
123     c += x * w;
124
125     if (last == w)
126         res.pop_back();
127     res.push_back({f, c});
128     last = w;
129 }
130
131 return res;
132 }
133 };

```

math.cpp

```

1 using ll = long long;
2 using ull = unsigned long long;
3
4 ll smod(ll x, ll m) {
5     return (x % m) < 0 ? x + m : x;
6 }
7
8 pair<ll, ll> igcd(ll a, ll b) {
9     a = smod(a, b);
10    if (!a)
11        return {b, 0};
12    ll s = b, t = a, x = 0, y = 1;
13    while (t) {
14        ll q = s / t;
15        s -= t * q;
16        x -= y * q;
17        swap(s, t);
18        swap(x, y);
19    }
20    if (x < 0)
21        x += b / s;
22    return {s, x};
23 }
24
25 ll pow_mod(ll x, ll n, int m) {
26     ll r = 1 % m;
27     x = smod(x, m);
28     for (; n; n >>= 1, x = x * x % m)
29         if (n & 1)
30             r = r * x % m;
31     return r;
32 }
33
34 ll inv_mod(ll x, ll m) {
35     return igcd(x, m).second;
36 }
37
38 pair<ll, ll> crt(const vector<ll>& r,
39     const vector<ll>& m) {
40     ll R = 0, M = 1;
41     for (int i = 0; i < (int)r.size(); i++) {
42         ll x = smod(r[i], m[i]);
43         auto [g, im] = igcd(M, m[i]);
44         ll u = m[i] / g;
45         if ((x - R) % g)
46             return {0, 0};
47         ll t = (__int128)smod((x - R) / g, u) *
48             im % u;
49         R += M * t;
50         M *= u;
51         R = smod(R, M);
52     }
53     return {R, M};
54 }
55
56 ll floor_sum(ll n, ll m, ll a, ll b) {
57     ull z = 0;
58     if (a < 0) {
59         ull x = smod(a, m);
60         z -= 1ULL * n * (n - 1) / 2 *
61             ((x - a) / m);
62         a = x;
63     }
64     if (b < 0) {
65         ull x = smod(b, m);
66         z -= 1ULL * n * ((x - b) / m);

```

math.cpp (cont.)

```

67     b = x;
68 }
69 for (;;) {
70     if (a >= m)
71         z += 1ULL * n * (n - 1) / 2 * (a / m),
72         a %= m;
73     if (b >= m)
74         z += 1ULL * n * (b / m), b %= m;
75     ull y = (ull)a * n + b;
76     if (y < (ull)m)
77         return (ll)z;
78     n = y / m;
79     b = y % m;
80     swap(a, m);
81 }
82 }

```

modint_convolution.cpp

```

1 struct mint {
2     static const int M = 998244353;
3     int x;
4     mint(long long v = 0)
5         : x((v % M + M) % M) {}
6     int val() const { return x; }
7     static int mod() { return M; }
8     mint& operator+=(mint a) {
9         if ((x += a.x) >= M)
10             x -= M;
11         return *this;
12     }
13     mint& operator--(mint a) {
14         if ((x -= a.x) < 0)
15             x += M;
16         return *this;
17     }
18     mint& operator*=(mint a) {
19         x = (long long)x * a.x % M;
20         return *this;
21     }
22     mint pow(long long n) const {
23         mint a = *this, r = 1;
24         for (; n >= 1, a *= a)
25             if (n & 1)
26                 r *= a;
27         return r;
28     }
29     mint inv() const { return pow(M - 2); }
30     mint& operator/=(mint a) {
31         return *this *= a.inv();
32     }
33     friend mint operator+(mint a, mint b) {
34         return a += b;
35     }
36     friend mint operator-(mint a, mint b) {
37         return a -= b;
38     }
39     friend mint operator*(mint a, mint b) {
40         return a *= b;
41     }
42     friend mint operator/(mint a, mint b) {
43         return a /= b;
44     }
45 };
46 using modint998244353 = mint;
47 void ntt(vector<modint998244353>& a, bool inv = 0) {
48     int n = a.size();

```

modint_convolution.cpp (cont.)

```

49     for (int i = 1, j = 0; i < n; i++) {
50         int b = n >> 1;
51         for (; j & b; b >>= 1)
52             j ^= b;
53         j ^= b;
54         if (i < j)
55             swap(a[i], a[j]);
56     }
57     for (int l = 2; l <= n; l <= 1) {
58         mint w = mint(3).pow((mint::M - 1) / l);
59         if (inv)
60             w = w.inv();
61         for (int i = 0; i < n; i += l) {
62             mint z = 1;
63             for (int j = 0; j < l / 2; j++) {
64                 mint x = a[i + j],
65                     y = a[i + j + l / 2] * z;
66                 a[i + j] = x + y;
67                 a[i + j + l / 2] = x - y;
68                 z *= w;
69             }
70         }
71     }
72     if (inv) {
73         mint z = mint(n).inv();
74         for (auto& x : a)
75             x *= z;
76     }
77 }
78 vector<modint998244353> convolution(
79     vector<modint998244353> a, vector<modint998244353> b) {
80     if (a.empty() || b.empty())
81         return {};
82     int n = a.size() + b.size() - 1, z = 1;
83     while (z < n)
84         z <= 1;
85     a.resize(z);
86     b.resize(z);
87     ntt(a);
88     ntt(b);
89     for (int i = 0; i < z; i++)
90         a[i] *= b[i];
91     ntt(a, 1);
92     a.resize(n);
93     return a;
94 }

```

geometry.cpp

```

1 using ld = long double;
2 using Point = complex<ld>;
3 using pt = Point;
4 const ld EPS = 1e-10L, PI = acosl(-1.L);
5
6 inline bool equal(const ld& a,
7     const ld& b) { //a,b が誤差を許して等しいか
8     return fabsl(a - b) < EPS;
9 }
10 inline int sgn(
11     const ld& x) { //x の符号を-1,0,1 で返す
12     return (x > EPS) - (x < -EPS);
13 }
14
15 Point unitVector(const Point&
16     a) { //a と同方向の単位ベクトル
17     assert(abs(a) > EPS);
18     return a / abs(a);
19 }

```

geometry.cpp (cont.)

```

20 Point normalVector(const Point&
21     a) { //a を反時計回りに 90 度回転
22     return a * Point(0, 1);
23 }
24 ld dot(
25     const Point& a, const Point& b) { //内積
26     return real(conj(a) * b);
27 }
28 ld cross(
29     const Point& a, const Point& b) { //外積
30     return imag(conj(a) * b);
31 }
32 Point rotate(const Point& p,
33     ld t) { //p を反時計回りに t(rad)回転
34     return p * polar<ld>(1, t);
35 }
36 ld radianToDegree(ld x) { //rad を度に変換
37     return x * 180 / PI;
38 }
39 ld degreeToRadian(ld x) { //度を rad に変換
40     return x * PI / 180;
41 }
42
43 struct Line {
44     Point a, b;
45     Line() = default;
46     Line(Point x, Point y)
47         : a(x), b(y) {
48     } //2 点 x,y を通る直線 (線分)
49     Line(
50         ld A, ld B, ld C) { //Ax+By=C を表す直線
51         assert(!equal(A, 0) || !equal(B, 0));
52         if (!equal(B, 0))
53             a = {0, C / B}, b = {1, (C - A) / B};
54         else
55             a = {C / A, 0}, b = {C / A, 1};
56     }
57 };
58
59 struct Circle {
60     Point p;
61     ld r;
62     Circle() = default;
63     Circle(Point x, ld y)
64         : p(x), r(y) {} //中心 x,半径 y の円
65 };
66
67 bool onLine(const Line& l,
68     const Point& p) { //p が直線 l 上か
69     return fabsl(cross(l.b - l.a, p - l.a)) <=
70         EPS;
71 }
72
73 bool onSegment(const Line& l,
74     const Point& p) { //p が線分 l 上か
75     return ccw(l.a, l.b, p) == 0;
76 }
77
78 Circle circumCircle(Point a, Point b,
79     Point c) { //三角形 abc の外接円
80     assert(fabsl(cross(b - a, c - a)) > EPS);
81     Point x = (a + b) / 2, y = (a + c) / 2;
82     Line l = {x, x + normalVector(b - a)};
83     Line m = {y, y + normalVector(c - a)};
84     Point p = crossPoint(l, m);
85     return {p, abs(p - a)};
86 }

```

geometry.cpp (cont.)

```

87
88 Point projection(const Line& l,
89     const Point&
90     p) { //l へ p から下ろした垂線の足
91     Point d = l.b - l.a;
92     assert(abs(d) > EPS);
93     return l.a +
94         d * dot(p - l.a, d) / norm(d);
95 }
96 Point reflection(const Line& l,
97     const Point& p) { //l に関する p の対称点
98     return 2.L * projection(l, p) - p;
99 }
100
101 int ccw(const Point& a, Point b,
102     Point c) { //a,b,c の位置関係を判定
103     b -= a;
104     c -= a;
105     ld x = cross(b, c);
106     if (abs(x) > EPS)
107         return x > 0 ? 1 : -1; //反時計/時計
108     if (dot(b, c) < -EPS)
109         return 2; //c-a-b
110     if (norm(b) + EPS < norm(c))
111         return -2; //a-b-c
112     return 0; //c が線分 ab 上
113 }
114
115 bool isOrthogonal(const Line& a,
116     const Line& b) { //2 直線が直交するか
117     return equal(
118         dot(a.b - a.a, b.b - b.a), 0);
119 }
120 bool isParallel(const Line& a,
121     const Line& b) { //2 直線が平行か
122     return equal(
123         cross(a.b - a.a, b.b - b.a), 0);
124 }
125 bool isIntersect(const Line& a,
126     const Line& b) { //2 線分が交差するか
127     return ccw(a.a, a.b, b.a) *
128         ccw(a.a, a.b, b.b) <=
129         0 &&
130         ccw(b.a, b.b, a.a) *
131         ccw(b.a, b.b, a.b) <=
132         0;
133 }
134
135 Point crossPoint(const Line& a,
136     const Line& b) { //2 直線の交点
137     Point d = a.b - a.a, e = b.b - b.a;
138     ld z = cross(d, e),
139         w = cross(d, b.a - a.a);
140     if (equal(z, 0)) {
141         assert(equal(w, 0));
142         return b.a;
143     }
144     return a.a + d * cross(b.a - a.a, e) / z;
145 }
146
147 ld distanceBetweenLineAndPoint(
148     const Line& l,
149     const Point& p) { //直線 l と点 p の距離
150     Point d = l.b - l.a;
151     assert(abs(d) > EPS);
152     return fabsl(cross(d, p - l.a)) / abs(d);
153 }

```

geometry.cpp (cont.)

```

154 ld distanceBetweenSegmentAndPoint(
155     const Line& l,
156     const Point& p) { //線分 l と点 p の距離
157     Point d = l.b - l.a;
158     assert(abs(d) > EPS);
159     if (dot(d, p - l.a) < EPS)
160         return abs(p - l.a);
161     if (dot(-d, p - l.b) < EPS)
162         return abs(p - l.b);
163     return fabsl(cross(d, p - l.a)) / abs(d);
164 }
165 ld distanceBetweenSegments(const Line& a,
166     const Line& b) { //2 線分間の距離
167     if (isIntersect(a, b))
168         return 0;
169     return min(
170         {distanceBetweenSegmentAndPoint(a, b.a),
171         distanceBetweenSegmentAndPoint(
172             a, b.b),
173         distanceBetweenSegmentAndPoint(
174             b, a.a),
175         distanceBetweenSegmentAndPoint(
176             b, a.b)});
177 }
178
179 ld SignedPolygonArea(const vector<Point>&
180     p) { //多角形の符号付き面積
181     ld s = 0;
182     for (int i = 0, n = p.size(); i < n; i++)
183         s += cross(p[i], p[(i + 1) % n]);
184     return s / 2;
185 }
186 ld PolygonArea(
187     const vector<Point>& p) { //多角形の面積
188     return fabsl(SignedPolygonArea(p));
189 }
190
191 bool isConvex(
192     const vector<Point>& p) { //多角形が凸か
193     if (p.size() < 3)
194         return false;
195     int s = 0;
196     for (int i = 0, n = p.size(); i < n;
197         i++) {
198         int t = sgn(cross(p[(i + 1) % n] - p[i],
199             p[(i + 2) % n] - p[(i + 1) % n]));
200         if (t && s && t != s)
201             return false;
202         if (t)
203             s = t;
204     }
205     return true;
206 }
207
208 int isContained(const vector<Point>& g,
209     const Point&
210     p) { //点 p が多角形 g の内/边上/外か (2/1/0)
211     bool in = 0;
212     for (int i = 0, n = g.size(); i < n;
213         i++) {
214         Point a = g[i] - p,
215             b = g[(i + 1) % n] - p;
216         if (imag(a) > imag(b))
217             swap(a, b);
218         if (imag(a) <= EPS && EPS < imag(b) &&
219             cross(a, b) < -EPS)
220             in ^= 1;

```

geometry.cpp (cont.)

```

221     if (equal(cross(a, b), 0) &&
222         dot(a, b) <= EPS)
223         return 1;
224     }
225     return in ? 2 : 0;
226 }
227
228 vector<Point> ConvexHull(vector<Point> p,
229     bool col =
230     false) { //凸包を反時計回りに返す(
231     ← col=true なら辺上も含む)
232     sort(p.begin(), p.end(),
233         [](Point a, Point b) {
234             return real(a) != real(b)
235                 ? real(a) < real(b)
236                 : imag(a) < imag(b);
237         });
238     p.erase(unique(p.begin(), p.end(),
239         [](Point a, Point b) {
240             return abs(a - b) < EPS;
241         }),
242         p.end());
243
244     int n = p.size(), k = 0;
245     if (n <= 1)
246         return p;
247     vector<Point> h(2 * n);
248     if (col) {
249         bool all = 1;
250         for (int i = 2; i < n; i++)
251             if (fabsl(cross(p[1] - p[0],
252                 p[i] - p[0])) > EPS)
253                 all = 0;
254         if (all)
255             return p;
256     }
257
258     auto bad =
259     [&](Point a, Point b,
260         Point c) { //凸包に不要な中間点か
261         ld x = cross(b - a, c - b);
262         return col ? x < -EPS : x < EPS;
263     };
264
265     for (auto x : p) {
266         while (
267             k >= 2 && bad(h[k - 2], h[k - 1], x))
268             --k;
269         h[k++] = x;
270     }
271     for (int i = n - 2, t = k + 1; i >= 0;
272         i--) {
273         while (k >= t &&
274             bad(h[k - 2], h[k - 1], p[i]))
275             --k;
276         h[k++] = p[i];
277     }
278     h.resize(k - 1);
279     return h;
280 }
281
282 int isIntersect(const Circle& a,
283     const Circle&
284     b) { //2 円の位置関係(返り値=共通接線数,
285     ← -1 は同一円)
286     ld d = abs(a.p - b.p), s = a.r + b.r,
287         t = fabsl(a.r - b.r);

```

geometry.cpp (cont.)

```

286 if (equal(d, 0) && equal(a.r, b.r))
287     return -1;
288 if (d > s + EPS)
289     return 4; //離れている
290 if (equal(d, s))
291     return 3; //外接
292 if (d < t - EPS)
293     return 0; //内包
294 if (equal(d, t))
295     return 1; //内接
296 return 2; //2点で交わる
297 }
298
299 Circle inCircle(const Point& a,
300 const Point& b,
301 const Point& c) { //三角形 abc の内接円
302     ld A = abs(b - c), B = abs(a - c),
303         C = abs(a - b);
304     assert(A + B + C > EPS &&
305         fabsl(cross(b - a, c - a)) > EPS);
306     Point p =
307         (a * A + b * B + c * C) / (A + B + C);
308     return {p,
309         distanceBetweenLineAndPoint({a, b}, p)};
310 }
311
312 vector<Point> crossPoint(const Circle& c,
313 const Line& l) { //円 c と直線 l の交点
314     ld d =
315         distanceBetweenLineAndPoint(l, c.p);
316     if (d > c.r + EPS)
317         return {};
318     Point h = projection(l, c.p);
319     if (equal(d, c.r))
320         return {h};
321     Point u = unitVector(l.b - l.a) *
322         sqrt(max<ld>(0, c.r * c.r - d * d));
323     return {h - u, h + u};
324 }
325
326 vector<Point> crossPoint(const Circle& a,
327 const Circle& b) { //2 円の交点
328     int m = isIntersect(a, b);
329     if (m == 4 || m == 0 || m == -1)
330         return {};
331
332     ld d = abs(b.p - a.p);
333     ld x = (a.r * a.r + d * d - b.r * b.r) /
334         (2 * d);
335     ld y =
336         sqrt(max<ld>(0, a.r * a.r - x * x));
337
338     Point u = (b.p - a.p) / d,
339         h = a.p + u * x;
340     if (m == 1 || m == 3)
341         return {h};
342
343     Point v = u * Point(0, y);
344     return {h + v, h - v};
345 }
346
347 vector<Point> tangentToCircle(
348 const Point& p,
349 const Circle& c) { //点 p から円 c への接点
350     ld d = norm(c.p - p);
351     if (d < c.r * c.r - EPS)
352         return {};

```

geometry.cpp (cont.)

```

353     return crossPoint(c,
354         Circle(
355             p, sqrt(max<ld>(0, d - c.r * c.r))));
356 }
357
358 vector<Line> tangent(const Circle& a,
359 const Circle& b) { //2 円の共通接線
360     vector<Line> r;
361     ld g = abs(a.p - b.p);
362     if (equal(g, 0))
363         return r;
364
365     Point u = unitVector(b.p - a.p),
366         v = normalVector(u);
367
368     for (int s : {-1, 1}) {
369         ld h = (a.r + b.r * s) / g,
370             q = 1 - h * h;
371         if (q < -EPS)
372             continue;
373
374         if (fabsl(q) < EPS) {
375             Point w = h > 0 ? u : -u;
376             r.push_back(
377                 {a.p + w * a.r, a.p + w * a.r + v});
378         } else {
379             Point U = u * h,
380                 V = v * sqrt(max<ld>(0, q));
381             for (int t : {1, -1}) {
382                 Point w = U + V * (ld)t;
383                 r.push_back({a.p + w * a.r,
384                     b.p - w * (b.r * s)});
385             }
386         }
387     }
388     return r;
389 }

```